

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

Servlets

En la plataforma Jakarta EE, la funcionalidad de generar dinámicamente páginas Web se puede lograr mediante dos vías: una es mediante el uso de Java Server Pages o JSP, y la otra con Servlets.

Los Servlets son muy efectivos para el desarrollo de aplicaciones basadas en Web, que ayudan a proveer acceso seguro a un sitio, además de interactuar con bases de datos, generar dinámicamente documentos HTML, XHTML o XML para ser desplegados en el cliente, y finalmente para mantener una única sesión de información para cada cliente.

La tecnología JSP es una extensión de la tecnología Servlet. Por lo general, las JSP se usan cuando la mayor parte del contenido que se envía al cliente es texto estático, y sólo una pequeña parte de ese contenido se genera dinámicamente con código Java. Y por lo común, los Servlets se usan en el caso inverso, es decir, cuando una pequeña porción del contenido a enviar es texto estático. De hecho, muchos Servlets no producen contenidos, y en vez de ello realizan alguna tarea y luego invocan a una página JSP para proveer una respuesta al cliente.

Los Servlets permiten implementar el modelo de comunicación entre clientes que realizan peticiones y servidores que las procesan y responden, vía el protocolo HTTP. Un cliente envía una petición HTTP al servidor o contenedor de Servlets, este recibe la petición y la redirecciona para ser procesada por el Servlet apropiado. Este Servlet la procesa eventualmente interactuando con una base de datos o con otros componentes del lado del servidor, tales como otros Servlets, alguna página JSP o bien otros componentes del servidor. El Servlet retorna el resultado al cliente, normalmente en forma de un documento para desplegar en el navegador, con formato HTML, XHTML, o XML, pero también puede generar y enviar archivos con otros formatos, como imágenes, archivos binarios, etc.

Esencialmente, un Servlet es una clase Java que implementa la interfaz Servlet del paquete jakarta.servlet. La interfaz Servlet provee cinco métodos, y algunos de ellos forman parte del llamado ciclo de vida del Servlet. Los métodos del ciclo de vida de un Servlet serán invocados automáticamente por el contenedor, en ciertos momentos de la ejecución del Servlet. Los cinco métodos son:

void init (ServletConfig)

Este método se invoca automáticamente sólo una vez, en el momento que el Servlet se carga en memoria en el contenedor. El parámetro de la clase ServletConfig es provisto en forma automática por el contenedor que ejecuta el Servlet.

ServletConfig getServletConfig ()

Este método retorna una referencia a un objeto que implementa la interfaz ServletConfig. Este objeto provee acceso a la información de configuración del Servlet, la cual incluye elementos tales como los parámetros de inicialización o el contexto del Servlet (un objeto ServletContext, que provee a su vez acceso al entorno de ejecución del Servlet, es decir, a su contenedor).

String getServletInfo ()

Este método es definido para retornar una cadena con información del Servlet, tal como su autor y su versión.

void service (ServletRequest, ServletResponse)

Este método es invocado por el contenedor para responder a una petición del cliente. Los parámetros de tipo ServletRequest y ServletResponse se usan para permitir acceso los flujos de entrada y salida respectivamente, para leer datos del cliente o enviar al mismo una respuesta. Estos flujos pueden ser basados en bytes o en caracteres. El método puede lanzar una excepción ServletException o IOException si algún problema ocurre durante la ejecución del Servlet.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

void destroy ()

Este método de “limpieza” es invocado cuando el Servlet termina su ejecución en el contenedor. Los recursos usados por el Servlet deberían ser liberados aquí (por ejemplo, archivos que el Servlet haya abierto deberían ser cerrados, al igual que alguna conexión con una base de datos, etc.)

El ciclo de vida de un Servlet comienza cuando el Servlet es cargado en memoria por el contenedor, normalmente como respuesta a la primera petición que el Servlet recibe. Antes de procesar esa primera petición, el contenedor invoca al método `init()`. Este método se invoca en este momento por única vez, y realiza tareas de inicialización. De aquí en adelante, `init()` no vuelve a invocarse, ni siquiera si el Servlet vuelve a ser requerido para procesar otra petición.

Cuando `init()` termina, el Servlet puede responder a la primera petición. Todas las peticiones son manejadas por el método `service()` del Servlet. Ese método recibe la petición, la procesa, y envía la respuesta al cliente. Durante el ciclo de vida del Servlet, el método `service()` será invocado una vez por cada petición que el Servlet reciba, y en cada petición se creará un hilo de ejecución nuevo dentro del cual se ejecutará `service()`.

Cuando el Servlet termina de ejecutarse, el contenedor invoca al método `destroy()` para liberar los recursos que haya usado.

Los paquetes para manejo de Servlets provistos por Java, definen dos clases abstractas que ya implementan la interfaz Servlet: ellas son la clase `GenericServlet` (del package `jakarta.servlet`) y la clase `HttpServlet` (del package `jakarta.servlet.http`). Estas clases proveen una implementación por default de todos los métodos de la interfaz Servlet, por lo que muchos Servlets sólo deben derivar de una de ellas y redefinir algunos de los métodos citados.

La primera de esas clases (`GenericServlet`) se usa para crear Servlets genéricos, independientes del protocolo de transferencia. Se trata normalmente de Servlets muy sencillos para los que la clase `GenericServlet` ya provee una implementación básica de los métodos `init()` y `destroy()`, quedando para el programador la redefinición del método `service()`.

La segunda clase (`HttpServlet`) define Servlets con capacidades de procesamiento más elaboradas y orientadas a peticiones mediante el protocolo HTTP. Una clase que derive de `HttpServlet` usualmente redefine alguno de los siguientes métodos provisto por `HttpServlet`:

- **doGet ()**: si el Servlet soporta peticiones con el método GET.
- **doPost ()**: Si el Servlet procesa la petición con POST.
- **doPut ()**: Para peticiones HTTP PUT.
- **doDelete ()**: Para peticiones HTTP DELETE.
- **init ()** y **destroy ()**: Para manejar recursos necesarios en el ciclo de vida del Servlet.
- **getServletInfo ()**: Para proveer información acerca del Servlet.

En la práctica, casi no hay razón para redefinir el método `service()` en una clase que derive de `HttpServlet`, ya que se implementación en `HttpServlet` maneja las peticiones HTTP estándar enviándolas a los métodos de gestión para cada tipo de petición HTTP (esto es, `service()` reenvía las peticiones a los métodos `doGet()`, `doPost()`, `doPut()` o `doDelete()` de la clase que deriva de `HttpServlet`). La clase `HttpServlet` también provee otros métodos como `doOptions()` o `doTrace()`, pero al igual que con `service()`, casi no hay motivos prácticos para tener que redefinirlos.

Los Servlets típicamente se ejecutan en contenedores multihilo, por lo que se debe cuidar que cada Servlet maneje peticiones concurrentemente, y ser cuidadosos para sincronizar el acceso a recursos compartidos. Los recursos compartidos que deberían sincronizarse son variables de instancia o de clase, archivos, conexiones a bases de datos y conexiones de redes.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

Una aplicación Web basada en Servlets típica, normalmente hará que los Servlets deriven de `HttpServlet`. Esta clase redefine el método `service()` de forma que pueda distinguir entre los tipos clásicos de peticiones recibidas desde un cliente: GET y POST. Los métodos `doGet()` y `doPost()` son usados por la clase para responder a cada una de las formas de petición citadas. Ambos son automáticamente invocados por `service()`, el cual primero determina de qué tipo de petición se trató, y luego a uno u otro para manejarla. Ese es el motivo por el cual no se recomienda redefinir el método `service()` en la clase que derive de `HttpServlet`: al hacerlo, el método podría no saber cómo distinguir si la petición fue GET o POST.

Los métodos `doGet()` y `doPost()` reciben dos argumentos: un objeto de la clase `HttpServletRequest` y otro de la clase `HttpServletResponse`, que permiten la interacción entre el cliente y el servidor. El primer parámetro permite realizar un acceso sencillo a los datos provistos en la petición, y el segundo permite un fácil retorno de resultados desde el Servlet hacia el cliente web.

La interfaz `HttpServletRequest` provee métodos para facilitar el acceso a los datos enviados en la petición. El contenedor que ejecuta el Servlet crea un objeto de una clase que haya implementado esta interfaz y se lo envía al método `service()` del Servlet, el cual a su vez lo envía a `doGet()` o a `doPost()`. Algunos de los métodos más útiles de esta interfaz son:

String getParameter (String)

Obtiene el valor de un parámetro enviado al Servlet como parte de una petición HTTP. El argumento `name` representa el nombre del parámetro.

Enumeration getParameterNames ()

Retorna los nombres de todos los parámetros enviados al Servlet como parte de una petición post.

String[] getParameterValues (String)

Para un parámetro con valores múltiples, el método retorna un arreglo de Strings con los valores del parámetro indicado.

Cookie[] get_cookies ()

Retorna un arreglo de objetos de la clase `Cookie` almacenados en el cliente por el servidor. Una cookie puede ser usada únicamente para identificar clientes en el Servlet.

HttpSession getSession (boolean)

Retorna un objeto de la clase `HttpSession` asociado con la sesión actual del cliente. Un objeto `HttpSession` puede ser creado por este método (argumento igual a `true`) si no existe en forma previa un objeto de esa clase para ese cliente. Los objetos de la clase `HttpSession` pueden ser usado en forma similar a los objetos `Cookie`, sólo para identificar clientes.

En forma similar, la interfaz `HttpServletResponse` provee métodos para facilitar la formulación de la respuesta al cliente. El contenedor que ejecuta el Servlet crea un objeto de una clase que haya implementado esta interfaz y se lo envía al método `service()` del Servlet, el cual a su vez lo envía a `doGet()` o a `doPost()`. Algunos de los métodos más útiles de esta interfaz son:

void addCookie (Cookie)

Se usa para agregar una `Cookie` a la cabecera de la respuesta al cliente. El tamaño máximo de la cookie, así como el permiso para grabar esa cookie en el cliente, determinan finalmente si la cookie podrá almacenarse o no en el cliente.

ServletOutputStream getOutputStream ()

Obtiene un flujo de salida, basado en bytes, para enviar datos binarios al cliente.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

PrintWriter getWriter ()

Obtiene un flujo de salida, basado en caracteres, para enviar datos de texto al cliente.

void setContentType (String)

Especifica el tipo MIME de la respuesta al navegador. El tipo MIME ayuda al navegador a determinar cómo mostrar los datos (o posiblemente a determinar qué otra aplicación ejecutar para procesar esos datos). Por ejemplo, el tipo MIME "text/html" indica que la respuesta es un documento HTML, y por lo tanto el navegador puede mostrar la página.

Cookies

En muchas ocasiones requeriremos mantener cierta información del usuario para poder mejorar su navegación. Sin embargo la personalización en la navegación trae aparejado un problema importante, conocido como **invasión de privacidad (privacy invasion)**: la organización dueña de un sitio Web accedido por el usuario, podría usar en forma no autorizada los datos que el usuario haya provisto (por ejemplo: esa organización podría vender esos datos a otras empresas del mismo rubro, o podría publicar esos datos en otros sitios, o podría usar números tarjetas de crédito para autorizar compras y gastos que el usuario no pidió, entre otras situaciones peligrosas).

Parte del problema es que el protocolo HTTP usado como soporte del mecanismo de petición/respuesta en la Web, es un protocolo *stateless* (desprovisto de información de estado): no soporta información persistente que permita que el servidor Web determine si una consulta pertenece a un cliente particular. Esto significa que, desde el punto de vista del servidor, todas las consultas podrían haber sido realizadas por el mismo cliente, o cada una podría haber sido hecha por clientes distintos. El servidor no tiene forma de distinguir entre estos casos usando sólo el protocolo HTTP.

Sin embargo, de alguna forma el servidor debería poder identificar a cada cliente si se quiere personalizar la forma en que cada uno navega. Hay varias formas muy conocidas para hacer esto:

- **Mediante el uso de cookies:** Una cookie es un pequeño archivo de texto que el Servlet almacena en la computadora del usuario, para ser recuperado más tarde en la misma sesión o en alguna sesión posterior. Una cookie podría almacenar, por ejemplo, un identificador único para ese usuario, generado por una aplicación en el servidor en el momento en que el usuario entró por primera vez al sitio.
- **Mediante seguimiento de sesión (session tracking):** Java provee un amplio soporte de información de sesión a través de la interfaz HttpServlet. Objetos de alguna clase que haya implementado esa interfaz pueden almacenar información de sesión que luego sea usada por el servidor.
- **Mediante el uso de campos ocultos:** el Servlet guarda información de la sesión en un form html usando campos tipo input marcados como ocultos (hidden). Cuando el cliente envía la información que se le solicitó, también viaja el valor de los campos ocultos.
- **Mediante la redefinición o sobreescritura de la URL:** el Servlet agrega información de seguimiento de sesión a modo de parámetro GET, directamente en la URL de los hipervínculos que el usuario podría seleccionar al navegar.

Respecto del uso de cookies, quizás sea esta la técnica más difundida de las cuatro. Una vez que el usuario realiza su primera identificación y el Servlet almacena una cookie con sus datos, esa cookie (o cookies si hubo varias) puede ser recuperada por el servidor la próxima vez que el

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

usuario acceda al sitio y realice otra petición. Técnicamente, una cookie es un conjunto de datos de texto que un Servlet incluye en un envío al cliente.

Cada interacción basada en HTTP incluye un encabezado (header) que contiene información acerca de la petición (si el envío se hizo desde el cliente al servidor) o acerca de la respuesta (si el envío fue del servidor al cliente). Cuando un objeto `HttpServletRequest` recibe una petición, el encabezado incluye información tal como el tipo de petición (GET o POST) y las cookies enviadas al servidor para obtener datos del usuario. Y cuando el servidor envía una respuesta al cliente, el encabezado incluye información como el formato de la respuesta (tipo MIME) y las cookies que el servidor necesita grabar en la computadora del usuario. Es importante aclarar que la posibilidad de permitir que se almacenen cookies en su computadora puede ser bloqueada por el usuario (el navegador puede estar configurado para no permitir que se graben cookies) En casos así, se debería informar al usuario que el sitio que está accediendo podría no funcionar correctamente.

Una cookie tiene una propiedad conocida como *maximum age* (duración máxima). Dependiendo del valor de esa propiedad, el navegador puede mantener la cookie mientras dure la sesión (o sea, hasta que el usuario cierre el navegador), o bien puede almacenar la cookie en la computadora del cliente para futuros usos. Cuando el navegador realiza una petición al servidor, las cookies previamente enviadas al cliente por el servidor son retornadas al servidor como parte de la petición realizada por el navegador. Una cookie es eliminada automáticamente cuando la cookie *expira* (o sea, cuando vence su *maximum age*).

El nombre de la cookie identifica a la cookie; y el valor de la cookie es la información asociada con la misma. Los navegadores que soportan cookies deben ser capaces de almacenar un mínimo de 20 cookies por sitio Web, y 300 cookies por usuario. En general, los navegadores limitan el tamaño de cada cookie a 4K por cookie (4096 bytes). Cada cookie almacenada en el cliente incluye un dominio, y el navegador envía la cookie sólo al dominio descrito en la cookie.

Por defecto, las cookies existen sólo para la sesión actual del navegador (hasta que el usuario cierra el navegador) Para hacer que una cookie persista más allá de la sesión actual, debe invocarse el método `setMaxAge()` de la clase `Cookie`, para indicar el número de segundos hasta que la cookie expire.

El método `addCookie()` de la interfaz `HttpServletResponse`, agrega la cookie al encabezado de la respuesta, y la misma es enviada al cliente como parte de ese encabezado. La información de cabecera siempre es provista primero al cliente, por lo que las cookies deberían ser agregadas a la respuesta con `addCookie()` antes que cualquier dato sea escrito como parte de la respuesta.

Para recuperar las cookies desde el cliente, debe utilizarse el método `getCookies()` de la interfaz `HttpServletRequest`, el cual retorna un arreglo de objetos de la clase `Cookie`. Cuando una operación GET o POST es realizada para invocar a un Servlet, las cookies asociadas con el ese servidor son automáticamente enviadas al Servlet. Si el método `getCookies()` no retorna null (cosa que hará, por ejemplo, si no había cookies), se usa el método `getName()` para obtener el nombre de cada cookie, y el método `getValue()` para obtener su valor. La clase `Cookie` provee varios métodos para manejar un objeto `Cookie`:

String getComment ()

Retorna un String describiendo el propósito de la cookie (**null** si no se fijó ningún comentario mediante el método `setComment()`).

String getDomain ()

Retorna una String que contiene el dominio de la cookie. Este determina que servidores pueden recibir la cookie. Por defecto, las cookies son enviadas al servidor que originalmente envió la cookie al cliente.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

int getMaxAge ()

Retorna un entero que representa la máxima duración de la cookie en segundos.

String getName ()

Retorna un String con el nombre de la cookie, tal como configurado por el constructor.

String getPath ()

Retorna un String con el prefijo URL para la cookie.

boolean getSecure ()

Retorna un boolean indicando si la cookie debería ser transmitida usando un protocolo seguro (valor **true**).

String getValue ()

Retorna String conteniendo el valor de la cookie, tal como fue fijado por el constructor o por setValue().

int getVersion ()

Retorna un entero con la versión del protocolo usado para crear la cookie. El valor 0 (default) indica el protocolo original tal como fue definido por NetScape. El valor 1 indica la versión actual, basada en *Request for Comments* (RFC) 2109.

void setComment (String)

El comentario describiendo el propósito de la cookie.

void setMaxAge (int)

Especifica la duración máxima de la cookie, en segundos.

void setPath(String)

Especifica el prefijo del URL "objetivo" que indica los directorios en los que el servidor puede recibir la cookie.

void setSecure (boolean)

Un valor true indica que la cookie debería enviarse sólo usando un protocolo seguro.

void setValue (String)

Fija el valor de la cookie.

void setVersion (int)

Fija el protocolo para la cookie.

Sesiones

Java provee soporte mejorado para el seguimiento de sesiones, mediante la interfaz HttpSession. Una pauta importante es que un Servlet no debe usar variables de instancia para mantener información de estado de un cliente, porque los clientes que accedan al Servlet en paralelo podrían sobrescribir las variables de instancia que compartan. Los Servlets deben mantener información de estado de los clientes en objetos de la clase HttpSession.

El método getSession() de la interfaz HttpServletRequest permite obtener el objeto HttpSession para el cliente. Si el servidor ya tenía un objeto de esa clase para el cliente (por una petición previa), el método getSession() retorna ese objeto. De otro modo, el valor true del argumento enviado a getSession() indica que el Servlet debe crear un objeto único HttpSession para ese cliente. Un valor falso causa que el método retorne null si el objeto no existía. Por lo tanto, un

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

valor false puede ayudar para saber si un cliente ya estaba autenticado dentro de una aplicación Web.

Al igual que una cookie, un objeto HttpSession puede almacenar pares <nombre/valor>. En la terminología de una sesión, estos elementos se llaman atributos y son colocados en un objeto HttpSession con el método setAttribute(). Uno de los principales beneficios de usar objetos de sesión en lugar de cookies es que un objeto de sesión puede almacenar cualquier objeto (y no sólo Strings) como valor de un atributo. Esto permite a los programadores Java mayor flexibilidad para determinar el tipo de información de estado que quisieran mantener para un cliente de sus aplicaciones.

Si un atributo con un nombre particular ya existe cuando setAttribute() es invocado, el objeto asociado con el nombre del atributo es reemplazado. Los pares <nombre/valor> agregados a un objeto HttpSession con setAttribute() permanecen disponibles hasta que la sesión actual del cliente en el navegador termine o hasta que la sesión sea explícitamente invalidada mediante una llamada al método invalidate() de HttpSession. Por otra parte, si el contenedor del Servlet es vuelto a arrancar, estos atributos pueden perderse.

La interfaz HttpSession proporciona los siguientes métodos:

String getId ()

Retorna el identificador de la sesión

Object getAttribute (String)

Retorna un objeto asociado al nombre que se especifica como parámetro.

Enumeration getAttributeNames ()

Retorna los nombres de los atributos que están almacenados en la sesión

int getMaxInactiveInterval ()

Retorna el tiempo máximo en el que la sesión quedará invalidada.

void setAttribute (String, Object)

Almacena un objeto en la sesión asociado al nombre especificado.

void removeAttribute (String)

Elimina de la sesión el objeto asociado al nombre especificado.

void setMaxInactiveInterval (int)

Especifica la duración máxima de la sesión, en segundos.

void invalidate()

Invalida la sesión.

Por defecto, el seguimiento de sesión utiliza cookies para asociar un identificador de sesión con un usuario. Para soportar también a los usuarios que acceden al Servlet con un navegador que no soporta cookies, o si este está programado para no aceptarlas, debemos utilizar reescritura de URL en su lugar.

Para ello, la interfaz HttpServletResponse proporciona los métodos encodeURL (String) y encodeRedirectURL(String). El primero se emplea para asociar el identificador de la sesión con la URL dentro de la página que se esté construyendo, mientras que el segundo se utiliza para asociar el identificador con la redirección a través del método sendRedirect(). Ambos métodos devuelven la nueva URL si la sobreescritura ha sido necesaria, o la misma URL si no ha sido necesario sobrescribir.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE SERVLETS, SESIONES Y COOKIES	VERSIÓN: 1.3 VIGENCIA: 29-04-2020

Si nuestro propósito es cambiar el comportamiento de seguimiento de sesión con cookies por sobreescritura de la URL en nuestra aplicación, podemos utilizar el listener `ServletContextListener` que al momento de inicializar el Servlet indique a través del método `setSessionTrackingModes(SessionTrackingMode)` que el seguimiento será por URL.